AsymAgentMark-TK: Weakly Asymmetric Behavioral Watermarking for LLM Agents

***Abstract*—Autonomous LLM agents increasingly make consequential multi-step decisions through tool calls, subgoal choices, and embodied actions. Existing content watermarks attribute generated text, but not the planning behavior that drives execution. AgentMark introduced distribution-preserving behavioral watermarking for this setting, but its symmetric construction requires the verifier to reconstruct the embedder's exact behavior distribution. This assumption is brittle: API quantization, temperature changes, model refreshes, and cross-model re-querying may preserve likely-behavior order while perturbing probabilities enough to break decoding.**

We propose AsymAgentMark-TK, a weakly asymmetric behavioral watermark for LLM agents. The embedder uses full per-step probabilities to preserve the agent's marginal policy, while the verifier needs only top-k rank order. The construction replaces probability-dependent differential bins with an interleaved binary partition tree over rank-sorted behaviors. Each selected behavior's rank determines a decoding path, enabling bit recovery from ranks alone. A fixed-budget HMAC-SHA512 DRBG schedule keeps both sides synchronized, and deterministic random linear network coding converts variable per-step bits into erasure-tolerant payload recovery.

We evaluate AsymAgentMark-TK on ALFWorld and ToolBench using DeepSeek and Gemini Flash. Across 11,820 trajectories, rank watermarking preserves task utility relative to a clean AgentMark pipeline: success is 78.6% versus 80.0% on ALFWorld and 77.1% versus 75.7% on ToolBench. Behavior-distribution divergence remains comparable to symmetric AgentMark. In the offline 8-bit decoder proxy, symmetric AgentMark has higher exact-channel recovery on ALFWorld, but under top-10 rank-only verification it drops from 0.700 to 0.014 while AsymAgentMark-TK remains at 0.068. These controlled proxy results show that partial channel knowledge can preserve useful provenance signal without exact probability reconstruction, while deployment attribution still requires longer registered payloads and live end-to-end recovery.

1. Introduction

LLM systems are rapidly shifting from passive text generators to agents that perceive context, plan, call tools, and act across multiple steps [1], [2], [3]. This shift changes what provenance must mean. For a chatbot, attributing the final string may be enough. For an agent, the consequential object is often the trajectory: which tool it selected, which subgoal it pursued, which API it invoked, or which embodied action it chose. These planning behaviors can affect safety, accountability, intellectual-property protection, and post-hoc auditing even when the final natural-language output is short or absent.

Content watermarking for LLM outputs has made substantial progress [4], [5], but it does not directly solve behavioral provenance. Planning behaviors are not token-native: an LLM's reasoning text may be compiled into structured tool calls or environment actions, discarding the lexical signal used by token watermarks. In response, AgentMark [6] formulated behavioral watermarking as distribution-preserving sampling over an explicit per-step behavior distribution. Its concrete construction, which we call AgentMark-F, embeds multi-bit identifiers while preserving the marginal behavior distribution, and it tolerates partial logs through erasure coding.

However, AgentMark-F is symmetric: decoding requires the verifier to reconstruct the same probability distribution used by the embedder. This is a strong assumption in real deployments. The verifier may access a rounded top-k API, a model after provider-side updates, a different serving stack, or a proxy model used for audit. These changes often preserve the relative order of plausible behaviors while shifting the numeric probabilities. Symmetric distribution-preserving encoders are fragile under this mismatch because their decoding bins are probability-dependent. A small change in probabilities can alter bin boundaries and desynchronize decoding.

This paper asks whether behavioral provenance can survive with less verifier knowledge. We answer affirmatively by importing the principle of weakly asymmetric steganography [7] into LLM-agent behavior channels. In our setting, the embedder has the full behavior distribution P_t, but the verifier observes only partial channel knowledge: the top-k rank ordering σ_t. This setting is weaker than exact-probability verification but stronger than fully asymmetric verification with no channel knowledge. It matches common audit interfaces that expose candidate rankings without calibrated probabilities.

We present AsymAgentMark-TK, a top-k rank-based behavioral watermark. The key technical idea is to redesign the encoder and decoder together. The encoder sorts behaviors by probability and recursively partitions the top-k list into even and odd ranks. At each level, a binary distribution-preserving primitive uses probability masses to choose a group without changing the marginal policy. The decoder, by contrast, never uses probability values: the selected behavior's rank determines the path through the

same interleaved partition tree, and odd branches reveal embedded bits. A fixed-budget DRBG schedule ensures that encoder and decoder consume the same pseudorandomness even when no bit is embedded at a level.

The claim is intentionally scoped. The paper establishes a channel-knowledge tradeoff: top-k ranks can preserve decoder signal when exact probabilities are unavailable. It does not claim attribution from selected actions alone, nor does it claim production certification from the offline 8-bit proxy tables. Audit-grade attribution still requires a pre-registered key/payload claim, sufficiently long payloads, calibrated rank reconstruction, and end-to-end RLNC recovery under the deployment logging stack.

Our contributions are:

- We formulate weakly asymmetric behavioral watermarking for LLM agents, separating the embedder's full channel access from the verifier's top-k rank-only access.
- We design AsymAgentMark-TK, a distribution-preserving rank encoder/decoder that decodes from top-k ordering rather than exact probabilities.
- We integrate the construction with deterministic random linear network coding (RLNC), enabling payload recovery from variable-bit and partially observed trajectories.
- We evaluate on ALFWorld [8] and ToolBench [9] across DeepSeek and Gemini Flash models, measuring utility, stealth, rank-only decoding, top-k sensitivity, rank noise, and false-positive accounting.

The rest of the paper proceeds as follows. Section II motivates behavioral watermarking and weak asymmetry. Section III defines the agent behavior channel, adversary, and verifier knowledge. Section IV presents the rank encoder, decoder, synchronization, and coding layer. Sections V and VI analyze security and evaluate utility, stealth, and rank-only recovery. Section VII discusses operational limits, Section VIII covers related work, Section IX covers validity and ethics, and Section X concludes.

2. Background and Motivation

2.1. Behavioral Watermarking for Agents

An LLM agent operates over a trajectory of observations, planning behaviors, and execution actions. At each step t, the agent chooses a behavior b_t from a finite admissible set $\mathcal{B}_t$. Examples include ALFWorld navigation actions, ToolBench API calls, or social-simulation behaviors in OASIS [10]. AgentMark models this choice as sampling from an elicited behavior distribution P_t and replaces ordinary sampling with a distribution-preserving encoder:

$$\hat{b}_t \leftarrow \mathsf{Enc}(P_t, r_t), \quad \Pr[\hat{b}_t = b] = P_t(b). \tag{1}$$

The distribution-preservation requirement is central. Agent trajectories are long-horizon and stateful; even small per-step distribution shifts can compound into task drift, failures, or visibly abnormal behavior.

2.2. The Symmetry Bottleneck

The concrete AgentMark-F construction uses differential recombination over P_t followed by cyclic-shift encoding. This is efficient when both embedder and verifier share exact probabilities. The same property makes it brittle under probability perturbation: if the verifier's reconstructed P'_t differs from P_t, recombined bins may differ, so the selected behavior is interpreted under the wrong bin structure.

The weakness is not merely numerical. Many verification interfaces intentionally withhold calibrated probabilities. Auditors may receive ranked candidates, top-k choices, or sampled logs. Even when probabilities are available, temperature scaling, quantization, floating-point differences, model upgrades, and fine-tuning can perturb values. These transformations frequently preserve a useful invariant: the order of high-probability behaviors.

This observation is especially natural for agents. Candidate behaviors are usually semantic alternatives produced by a planning prompt: a tool call, a navigation operation, a search/refine step, or a social action. A downstream auditor may be able to ask a model to rank plausible next behaviors from the same logged context even if the exact sampling distribution is hidden. In this sense, behavior-level verification has a useful middle ground that token-level watermarking often lacks: ranks are easier to reproduce than calibrated probabilities, and more informative than the selected action alone.

Table 1 summarizes the resulting distinction. AsymAgentMark-TK is not a replacement for text watermarking; it targets the behavior carrier that remains after plans are compiled into tool calls or environment actions. It is also not a capacity improvement over symmetric AgentMark when exact probabilities are available. Its purpose is to preserve AgentMark's behavior-policy guarantee while weakening the verifier's channel requirement.

2.3. Weak Asymmetry

Provably secure steganography traditionally separates symmetric settings, where sender and receiver share channel distributions, from asymmetric settings, where the receiver has no channel oracle [11], [12]. Weak asymmetry occupies the middle ground: the sender has the full channel, while the receiver has partial but nontrivial channel knowledge [7]. We instantiate this idea for behavioral watermarking. The embedder uses P_t to preserve the agent policy; the verifier uses only $\sigma_t = \mathrm{argsort}_k(P_t)$ to decode.

3. Problem Formulation

3.1. Agent Behavior Channel

For a trajectory of length T, each step t has a behavior set $\mathcal{B}_t = \{b_{t,1}, \ldots, b_{t,n_t}\}$ and a behavior distribution P_t

induced by the agent and environment context. A watermarking key K_{sh} and step context h_t derive per-step pseudorandomness. The embedder encodes a payload $m \in \{0,1\}^L$ into selected behaviors $\hat{b}_1, \ldots, \hat{b}_T$ while preserving P_t at each step.

The verifier observes a possibly incomplete set of indices $I \subseteq \{1, \ldots, T\}$ and, for each observed step, the selected behavior $\hat{b}_t$ plus channel side information. In the symmetric setting, the side information is P_t or an exact reconstruction. In our setting, it is the top-k ordering σ_t.

Verifier knowledge. We distinguish three verifier views. In the *exact view*, the verifier receives P_t and can run the same decoder as the embedder. In the *rank view*, the verifier receives only $\sigma_t^k = (b_{(1)}, \ldots, b_{(k)})$, the ordered list of the top-k behaviors. In the *selected-only view*, the verifier receives only $\hat{b}_t$ and no usable channel side information. The exact view is a useful upper bound; the rank view is the deployment target of AsymAgentMark-TK; the selected-only view is a negative control showing that behavioral provenance cannot be recovered from actions alone without some channel side information.

Rank consistency. For two distributions P_t and P'_t over the same candidate list, define top-k rank consistency as

$$\mathrm{argsort}_k(P_t) = \mathrm{argsort}_k(P'_t). \tag{2}$$

This condition permits arbitrary probability-value changes as long as the verifier preserves the ordered top-k set. It covers monotone transformations such as temperature rescaling, but it also covers less structured API changes whose empirical effect is to leave ranks unchanged. When the equality fails, we measure degradation through rank-noise experiments rather than assuming correctness.

3.2. Adversary and Requirements

We consider three perturbations. First, *step erasure*: some logged steps are missing due to execution failures, platform deletion, or incomplete telemetry. Second, *distribution perturbation*: a verifier reconstructs P'_t whose numeric values differ from P_t. Third, *rank noise*: the verifier's top-k order disagrees with the embedder's order, modeling top-k instability or proxy-model disagreement.

The adversary observes watermarked trajectories and may know the algorithm and public parameters, including k, but not K_{sh}. This is the standard secret-key provenance setting: a platform or model owner embeds an identifier, while a later verifier checks a claimed key and payload. We do not rely on security through obscurity of the rank partition or coding scheme.

The adversary has two goals. A *removal* adversary tries to make a genuinely watermarked trajectory verify as inconclusive or negative by deleting steps, perturbing ranks, or changing behavior serialization. A *framing* adversary tries to make an unwatermarked or differently watermarked trajectory verify under another party's claim. Claim binding and false-positive thresholds address framing when the key is secret; arbitrary trajectory rewriting, compromised keys, and compromised logging infrastructure are outside the cryptographic claim and are handled as deployment assumptions.

A scheme should satisfy:

- **Utility preservation**: $\Pr[\hat{b}_t = b] = P_t(b)$ per step.
- **Weak-asymmetric correctness**: if $\mathrm{argsort}_k(P'_t) = \mathrm{argsort}_k(P_t)$, decoding from P'_t's order returns the same bits as decoding from P_t's order.
- **Low false positives**: a clean or unrelated trajectory should recover a pre-registered claimed L-bit payload with probability approximately 2^{-L} when decoded bits are independent of the claim.
- **Erasure tolerance**: payload recovery should degrade gracefully when only a subset of steps is observed.

Detection statistic. For an audited trajectory, the verifier decodes a multiset of coded bits and attempts to recover payload $\hat{m}$. A strict positive attribution requires $\hat{m} = m$ for a pre-registered claimed payload. For bit-level channel diagnostics, we also report the bit match rate between decoded coded bits and the coded stream induced by m. Payload recovery is stricter and is used for provenance claims; bit match is useful for comparing noisy channels even when there are too few equations for full RLNC recovery.

3.3. Non-Goals

AsymAgentMark-TK is not a general-purpose watermark removal defense. If an adversary controls the agent and can arbitrarily rewrite the trajectory, no behavioral watermark can force attribution without an external logging root of trust. Our target is post-hoc verification for trajectories that are generated by an honest embedder but later audited through an imperfect channel interface. We also do not assume the verifier can re-run the exact original model. The verifier may use a same-family model, a refreshed endpoint, or a restricted API as long as the top-k order remains sufficiently stable.

4. Design

4.1. Overview

AsymAgentMark-TK has four components, summarized in Fig. 1. First, the agent elicits or estimates P_t over candidate behaviors, following AgentMark's black-box-compatible behavior-distribution interface [6]. Second, the encoder sorts behaviors by descending probability and restricts embedding to the top-k region. Third, recursive interleaved binary splitting embeds bits while preserving the behavior distribution. Fourth, deterministic RLNC maps the payload into a stream of coded bits so that the verifier can solve for the payload after observing enough independent coded bits.

4.2. Binary Distribution-Preserving Primitive

At a tree node, the encoder must choose between two groups with masses S_0 and S_1 while optionally embedding

Embedder: full channel	$\rightarrow$	**Trajectory log**	$\rightarrow$	**Verifier: partial channel**
Agent context h_t and candidate behaviors $\mathcal{B}_t$. Elicit probabilities P_t and sort top-k order σ_t. Use P_t to sample a watermarked behavior with the same marginal distribution.		Logged behavior $\hat{b}_t$, step context, claimed payload identifier, and audit metadata. The log may omit steps or be checked after model/API changes.		Reconstruct only the top-k order σ'_t from a restricted API or proxy model. Decode bits from the rank of $\hat{b}_t$; solve RLNC equations for the payload.

Figure 1. System view of weakly asymmetric behavioral watermarking. The embedder needs calibrated probabilities to preserve utility; the verifier needs only rank order for decoding, plus enough observed coded bits for RLNC recovery.

TABLE 1. POSITIONING AGAINST NEARBY WATERMARKING SETTINGS.

Scheme	Carrier	Preserved object	Verifier channel
Token watermarks	Text tokens	Text distribution	Generated text/statistic
AgentMark-F	Agent behaviors	Behavior policy P_t	Exact P_t
AsymAgentMark-TK	Agent behaviors	Behavior policy P_t	Top-k rank σ_t

one bit. Assume $S_0 \geq S_1$ and $S = S_0 + S_1$. Given message bit $m \in \{0, 1\}$ and $r \leftarrow U[0, 1)$ from the DRBG, define

$$r_m = (rS + \tfrac{1}{2}Sm) \bmod S. \tag{3}$$

BinEnc selects group 0 if $r_m < S_0$ and group 1 otherwise. If group 1 is selected, one bit is embedded; if group 0 is selected, no bit is consumed and the same bit is retried at the next level. Because r_m is uniform over $[0, S)$ for uniform r, the selected group has exactly the desired marginal probabilities S_0/S and S_1/S.

Why zero-bit branches are intentional. The primitive gives up a bit when the high-mass group is selected because forcing every branch to carry a bit would require reweighting the group probabilities. The deferred-bit design is the same kind of tradeoff used by distribution-preserving steganography: capacity is opportunistic, but the cover distribution is unchanged. In agent settings this is a feature rather than a cosmetic constraint, because utility regressions caused by shifted planning choices are easier for users and downstream systems to notice than small variations in textual style.

4.3. Rank Encoder

Let σ_t be the descending rank ordering of P_t. The encoder first samples whether to enter the top-k region with probability $S_{\text{top}} = \sum_{i=1}^{k} P_t(\sigma_t(i))$. If not, it samples from the tail proportionally and embeds no bits. Inside top-k, it normalizes probabilities and applies recursive splitting:

1) Split the current sorted list into even ranks $(0, 2, 4, \ldots)$ and odd ranks $(1, 3, 5, \ldots)$.
2) Use BinEnc on the two group masses.
3) Recurse into the selected group until a single behavior remains.

Interleaving is important. A top-half/bottom-half split would place most mass in the first group, producing low embedding opportunity. Even/odd splitting balances mass while preserving a rank path that the verifier can recover.

Algorithm 1: One-step rank encoding.
Input: distribution P_t, context h_t, key K_{sh}, payload stream M, pointer ℓ, top-k.
1. Derive DRBG $\mathcal{G}_t$ from (K_{sh}, h_t).
2. Sort behaviors: $\sigma_t \leftarrow \text{argsort}(P_t)$.
3. With probability $1 - \sum_{i \leq k} P_t(\sigma_t(i))$, sample from the tail and return no bits.
4. Normalize the top-k distribution D.
5. While $|D| > 1$: split D into even/odd ranks; run BinEnc on the group masses; consume a message bit only when BinEnc enters the odd group; recurse into the chosen group.
6. Advance unused DRBG calls until exactly $\lceil \log_2 k \rceil$ calls are consumed.
Output: selected behavior and consumed coded bits.

Figure 2. Encoder sketch. The encoder uses probability values only to preserve the marginal behavior distribution.

Algorithm 2: One-step rank decoding.
Input: selected behavior $\hat{b}_t$, top-k order σ_t, context h_t, key K_{sh}.
1. Derive the same DRBG sequence $r_0, \ldots, r_{\lceil \log_2 k \rceil - 1}$.
2. If $\hat{b}_t \notin \sigma_t[1:k]$, return the empty string.
3. Let q be the zero-indexed rank of $\hat{b}_t$.
4. While $q \neq 0$: if q is odd, append 1 when the corresponding $r_i < 0.5$ and 0 otherwise; set $q \leftarrow \lfloor q/2 \rfloor$.
Output: decoded coded bits for step t.

Figure 3. Decoder sketch. No numeric probability appears in the decoder.

4.4. Rank Decoder

Given selected behavior $\hat{b}_t$ and top-k order σ_t, the decoder finds the zero-indexed rank q of $\hat{b}_t$. If $\hat{b}_t$ is outside top-k, the step yields no bits. Otherwise, the decoder reconstructs the path by repeatedly reading the parity of q:

$$q_0 = q, \quad q_{\ell+1} = \lfloor q_\ell / 2 \rfloor. \tag{4}$$

Whenever q_ℓ is odd, the selected path entered the odd group and one bit was embedded. The value of that bit is determined by the same DRBG value used at level ℓ: the implementation decodes it as 1 if $r_\ell < 0.5$ and 0 otherwise. The decoder consumes exactly $\lceil \log_2 k \rceil$ DRBG values per step, matching the encoder's fixed synchronization budget.

4.5. Coding Layer

Per-step embedding capacity varies with P_t, k, and trajectory length. To avoid requiring a contiguous bitstream, AsymAgentMark-TK uses deterministic RLNC over GF(2),

following the standard random linear network coding principle [13]. For payload $m \in \{0,1\}^L$, the i-th coded bit is

$$c_i = \langle a_i, m \rangle \bmod 2, \tag{5}$$

where coefficient vector a_i is generated deterministically from a stream key and index i. The verifier collects decoded coded bits and solves the resulting linear system by Gaussian elimination over GF(2). This layer is inherited from AgentMark's erasure-resilience design but is independent of whether the step decoder is symmetric or rank-only.

The coding layer also cleanly separates two questions that are often conflated. The step decoder asks whether an observed behavior can yield a reliable coded bit under the verifier's channel view. RLNC asks whether enough independent coded bits have survived to solve for the payload. This separation is useful for weak asymmetry: a rank-only decoder may produce fewer bits than an exact-probability decoder, but the coding layer can still recover when the trajectory is long enough or the audit window spans enough steps.

4.6. Verification Protocol

An audit takes as input a pre-registered key/payload claim, a trajectory log, and a verifier-side procedure for reconstructing top-k candidate orders. The verifier first derives the per-step DRBG streams from logged contexts and decodes all observed steps using RankDec. It then runs RLNC elimination over the recovered coded equations. If the system recovers the registered payload and the reconstructed ranks pass a deployment-specific quality gate, the audit reports a positive attribution. If the payload does not recover but the rank-quality gate fails or too few independent equations are available, the audit reports *inconclusive*. A negative attribution is reserved for the case where the verifier has enough rank-consistent evidence to decode but the recovered payload contradicts the registered claim.

The rank-quality gate is intentionally separate from the watermark statistic. A verifier can estimate it using held-out calibration contexts, duplicate model queries, or agreement between independent rankers. This prevents a common operational error: treating a failed rank-only decode as evidence that a trajectory is clean when the verifier's channel reconstruction is itself unreliable.

For short trajectories, the verifier may not obtain enough independent RLNC equations for strict payload recovery. In that case, the bit-match statistic is reported only as supporting evidence with a binomial tail probability under the null model of random agreement. This diagnostic can trigger further collection or manual review, but it is not by itself a positive attribution decision.

4.7. Implementation Notes

The implementation is integrated through an `AgentWatermarker` SDK wrapper with `algorithm="rank"` as the default. Internally, `Dist` stores probability tensors and stable indices, `RankEncStep` applies the interleaved split, and `RankDecStep` reconstructs bits from sorted indices. The DRBG is an HMAC-SHA512 reseed construction inspired by Meteor [12]. A synchronization counter, `rt_sync`, forces both sides to advance exactly $\lceil \log_2 n \rceil$ random draws per decoded top-k step. This detail is small but critical: without it, a zero-bit level would shift all later decoding.

Tie handling uses stable sorting after probability normalization. Ties are rare in model-scored behavior sets but can occur after rounding or prompt templates that assign equal fallback scores. Stable sorting makes the order deterministic when both sides see the same candidate list. If the verifier obtains a tied but differently ordered top-k list, this is treated as rank noise and measured through the adjacent-swap experiment.

4.8. Parameter Selection

The top-k parameter controls a reliability–capacity tradeoff. Larger k exposes more leaves in the partition tree and can increase decoded length, but it also includes lower-probability tail behaviors whose relative ranks are less stable across models and serving conditions. Smaller k is more conservative and better aligned with restricted APIs, but it leaves less room for embedding. In our experiments we report $k \in \{2, 4, 6, 8, 10, 20\}$ rather than tuning to a single best value. A deployment can choose k from a calibration set by maximizing expected payload recovery subject to a minimum top-k rank agreement threshold.

5. Security Analysis

Proposition 1 (distribution preservation). For each step t and behavior $b \in \mathcal{B}_t$, AsymAgentMark-TK outputs b with probability $P_t(b)$.

Proof sketch. The top/tail gate selects the top-k region with probability equal to its total mass and otherwise samples the tail proportionally. Conditioned on entering top-k, each binary split uses BinEnc, whose selected group probability equals its normalized mass because r_m is uniform. Induction over the partition tree gives each leaf its normalized probability within top-k. Multiplying by the top-k mass recovers $P_t(b)$. □

Theorem 1 (weak asymmetry). Let P_t and P'_t satisfy $\mathrm{argsort}_k(P_t) = \mathrm{argsort}_k(P'_t)$. For any selected behavior $\hat{b}_t$ and shared context-derived DRBG stream,

$$\mathsf{RankDec}(\hat{b}_t, \mathrm{argsort}_k(P_t)) = \mathsf{RankDec}(\hat{b}_t, \mathrm{argsort}_k(P'_t)). \tag{6}$$

Proof. RankDec accesses channel information only through the rank position of $\hat{b}_t$ in the top-k order. If the two orders are equal, this rank is identical. The DRBG stream depends on the secret key and context, not probability values. Therefore the decoded path and bit values are identical. □

False positives. For an unwatermarked or unrelated trajectory, strict attribution requires recovery of a pre-registered claimed payload. Under the standard assumption that decoded clean bits are independent of that claim, an L-bit payload match occurs with probability 2^{-L}. In practice the verifier can tune L to the desired audit threshold; $L = 32$ yields 2.33×10^{-10} and $L = 64$ yields 5.42×10^{-20}.

Why symmetric decoding fails under rank-only views. Differential recombination constructs bins from probability gaps. Two distributions with the same rank ordering can have different gaps and therefore different bins. A behavior selected from the embedder's bin can land in a different verifier bin, causing both bit errors and variable-length desynchronization. Theorem 1 avoids this failure mode by making the decoder's sufficient statistic exactly the rank path, not the probability mass assigned to that path.

Key privacy. The watermark key is used only to derive context-specific pseudorandomness. The public trajectory and rank list do not reveal the key under the pseudorandom function assumption for HMAC-SHA512. Reusing the same key across many trajectories is therefore acceptable for provenance verification, though operational deployments should still rotate keys across tenants or products to limit blast radius if a key is disclosed.

Claim binding. A watermark match is meaningful only for a claimed key and payload that were bound to an owner before the audit. Otherwise, an operator could test many keys or payloads after observing a trajectory and inflate the effective false-positive rate. Deployments should therefore register a key identifier, payload namespace, and issuance time in an append-only registry or signed manifest. The verifier can then count only pre-registered claims and apply the multiple-testing correction in Section 6.12.

Limits. AsymAgentMark-TK is robust to value perturbations that preserve top-k order, not to arbitrary rank manipulation. If an adversary can force targeted rank swaps in the top-k list, it can corrupt decoded paths. This is the expected boundary of any rank-only weakly asymmetric scheme. The evaluation therefore includes adjacent rank-noise injection.

6. Evaluation

6.1. Setup

We evaluate on ALFWorld [8] and ToolBench [9]. ALFWorld tests embodied planning over ID and OOD splits; ToolBench tests tool-use behavior across six split categories. We use DeepSeek [14] and Gemini Flash [3]. The post-processed corpus contains 11,820 trajectories and 4,728 watermark decode-capable trajectories. Methods are: vanilla LLM agent, clean AgentMark pipeline without embedding, red-green (RG) behavior bias, symmetric AgentMark-F, and rank-based AsymAgentMark-TK.

Unless otherwise stated, recovery rates are computed from the available offline decoder artifacts. Capacity tables are bit-level channel proxies over logged trajectories and should not be conflated with a separate end-to-end RLNC recovery run. We use 8-bit payloads in these tables to make capacity differences measurable on short benchmark trajectories; deployable attribution should use longer payloads and the false-positive accounting in Section 6.12.

TABLE 2. EVALUATION COVERAGE AFTER POSTPROCESSING.

Dataset	Model/split groups	Completed runs	Decode-capable trajectories
ALFWorld	4	3/3 per group	3,288 (1,644 per watermark)
ToolBench	2	18/18 per group	1,440 (720 per watermark)

Table 2 summarizes the evaluation coverage. Utility and stealth metrics use all five methods. Decode-capable counts apply to the two watermarking methods because vanilla, clean, and RG trajectories do not carry payload metadata. The two watermarked methods are balanced within each dataset, which avoids comparing rank and symmetric decoders on different trajectory pools.

6.2. Evaluation Protocol

The evaluation pipeline starts from logged trajectories containing the selected behavior, candidate list, behavior probabilities when available, and watermark metadata. Utility and JSD are computed over all valid task trajectories. Watermark recovery is computed on the decode-capable subset: trajectories with a recorded watermark configuration, a recoverable candidate ordering, and enough logged context to replay the deterministic DRBG schedule. This distinction is reported explicitly in the corpus counts above.

For each decode-capable trajectory, we run the same verifier under several channel views. The *exact* view uses the logged probabilities as an upper bound. The *top-k* views discard numeric probabilities and retain only the ordered candidate list. The selected-only view is used as a negative control when available. Rank-noise experiments use a bit-level disagreement proxy parameterized by ϵ because the postprocessed logs do not contain full alternate rank lists for every verifier; erasure experiments remove a random fraction of observed steps before RLNC solving. All tables aggregate over the available model/split groups rather than selecting the best configuration per task.

6.3. Artifact Availability

The artifact bundle is organized so that each reported number has a concrete source. The paper package contains the manuscript, resolved bibliography, compiled PDF, and build helper. The rank encoder and decoder are implemented in `watermark_sampler.py`, and deterministic RLNC is implemented in `rlnc_codec.py`. In the `week2_postprocess` result directory, `stage_b_2_1_utility.csv` supports Table 3, `stage_b_2_2_jsd.csv` supports Table 4, `stage_b_1_1_capacity_proxy.csv` supports Table 5, `stage_b_1_2_topk_ablation.csv` supports Table 6, `stage_c_3_1_rank_noise.csv` supports

Table 7, and `stage_c_3_2_erasure_channel.csv` supports Table 8. The false-positive and confidence artifacts, `stage_c_3_4_false_positive.csv` and `stage_d_4_1_confidence_curve.csv`, support Section 6.12. The accompanying summary JSON records aggregate counts used for Table 2. The package README records the full repository paths. We treat these artifacts as a reproducibility trail for the reported postprocessed results; a live end-to-end rerun is a separate deployment study rather than evidence used for the present capacity tables.

6.4. Research Questions

The evaluation answers six questions. **RQ1**: Does rank-based embedding preserve task utility compared with clean and symmetric AgentMark pipelines? **RQ2**: Does it leave behavior-frequency distributions close to clean trajectories? **RQ3**: What is the tradeoff between exact-probability capacity and rank-only robustness? **RQ4**: How sensitive is the method to the top-k parameter? **RQ5**: How quickly does decoding degrade when the verifier's rank order is noisy? **RQ6**: How does recovery change when trajectory steps are missing?

6.5. Metrics

We report task success rate and average steps for utility. We report Jensen-Shannon divergence (JSD) between empirical action-frequency distributions for a coarse stealth check. For watermark recovery, *payload recovery* is the fraction of trajectories whose decoded payload exactly matches the embedded payload; in the offline proxy tables this is the 8-bit benchmark payload defined in the setup, not an audit-grade deployment identifier. *Bit match* is the average agreement between decoded coded bits and the expected coded stream. When a verifier view recovers no coded bits, both payload recovery and bit match are reported as zero. The latter is not a substitute for attribution, but it is a sensitive diagnostic for channel degradation. For confidence analysis, we use a one-sided binomial tail probability under the null hypothesis that each decoded bit matches the claim with probability $1/2$.

6.6. Utility Preservation

Table 3 reports average task success and steps. On ALFWorld, AsymAgentMark-TK reaches 78.6% success versus 80.0% for clean and 79.4% for AgentMark-F. On ToolBench, AsymAgentMark-TK reaches 77.1%, slightly above the clean pipeline's 75.7%. The result supports the distribution-preservation goal: rank encoding does not introduce the large utility tax associated with naive behavior bias.

6.7. Behavioral Stealth

We compare empirical action distributions against clean trajectories using Jensen-Shannon divergence (JSD).

TABLE 3. TASK UTILITY AVERAGED OVER AVAILABLE MODEL/SPLIT GROUPS. RG DENOTES RED-GREEN BEHAVIOR BIAS.

Dataset	Method	Success	Steps	Groups
ALFWorld	Vanilla	69.1	16.54	4
	Clean	80.0	16.23	4
	RG	74.9	17.58	4
	AgentMark-F	79.4	16.58	4
	AsymAgentMark-TK	78.6	16.40	4
ToolBench	Vanilla	84.1	4.06	2
	Clean	75.7	5.32	2
	RG	75.1	4.78	2
	AgentMark-F	74.7	4.54	2
	AsymAgentMark-TK	77.1	5.07	2

TABLE 4. MEAN JSD TO CLEAN BEHAVIOR FREQUENCY. LOWER IS CLOSER TO CLEAN.

Dataset	AgentMark-F	AsymAgentMark-TK	RG	Vanilla
ALFWorld	0.122	0.121	0.134	0.564
ToolBench	0.174	0.140	0.181	0.162

On ALFWorld, mean JSD to clean is 0.121 for AsymAgentMark-TK and 0.122 for AgentMark-F; on ToolBench it is 0.140 for AsymAgentMark-TK and 0.174 for AgentMark-F. RG is comparable on ALFWorld but worse on ToolBench. These measurements are coarse because behavior distributions are task- and split-dependent, but they show no obvious distributional anomaly introduced by rank splitting.

6.8. Rank-Only Verification

Table 5 compares payload recovery and bit match for 8-bit payloads under different verifier knowledge. Under exact probabilities on ALFWorld, AgentMark-F has higher payload recovery (0.700) than AsymAgentMark-TK (0.532), reflecting the efficiency cost of rank-only structure. Under top-10 rank-only verification, AgentMark-F collapses to 0.014 payload recovery, while AsymAgentMark-TK remains at 0.068. This is the central tradeoff: AgentMark-F is better when exact probabilities are available; AsymAgentMark-TK is more reliable when the verifier only has rank knowledge.

ToolBench is lower-capacity for both schemes because trajectories are short and many tool choices have low recoverable entropy. Nevertheless, AsymAgentMark-TK remains stable from exact probabilities to top-10 rank-only decoding (0.033 to 0.032), whereas AgentMark-F drops from 0.085 to 0.003.

The selected-only column is the negative control: when the verifier sees only the executed behavior and no candidate ordering, neither scheme recovers coded bits. This validates the weak-asymmetric framing. AsymAgentMark-TK weakens exact probability knowledge to rank knowledge; it does not claim attribution from the action log alone.

TABLE 5. OFFLINE 8-BIT DECODER PROXY UNDER VERIFIER CHANNEL KNOWLEDGE. CELLS SHOW PAYLOAD RECOVERY / BIT-MATCH RATE; SEL. ONLY IS THE SELECTED-BEHAVIOR-ONLY NEGATIVE CONTROL.

Dataset	Method	Exact	Top-10	Top-6	Top-2	Sel. only
ALFWorld	AgentMark-F	.700/.985	.014/.594	.004/.498	.004/.504	.000/.000
	AsymAgentMark-TK	.532/.942	.068/.718	.028/.641	.006/.566	.000/.000
ToolBench	AgentMark-F	.085/.582	.003/.299	.000/.292	.000/.309	.000/.000
	AsymAgentMark-TK	.033/.502	.032/.501	.007/.470	.000/.357	.000/.000

TABLE 6. TOP-k ABLATION FOR ASYMAGENTMARK-TK IN THE OFFLINE DECODER PROXY. CELLS SHOW BIT-MATCH / PAYLOAD RECOVERY.

Dataset	$k=2$	$k=4$	$k=6$	$k=8$	$k=10$	$k=20$
ALFWorld	.558/.006	.601/.013	.642/.028	.670/.048	.707/.064	.863/.313
ToolBench	.351/.000	.426/.006	.470/.010	.488/.021	.495/.032	.505/.035

6.9. Top-k Ablation

Increasing k raises both the amount of available rank information and the risk of rank instability. In ALFWorld, AsymAgentMark-TK's mean bit-match rate rises from 0.558 at $k = 2$ to 0.863 at $k = 20$, with payload recovery increasing from 0.006 to 0.313. ToolBench improves more modestly, from 0.351 to 0.505 bit-match, reflecting lower behavior entropy and shorter horizons. These results suggest that k should be selected per environment: larger k helps high-entropy embodied planning, while low-entropy tool tasks require either longer traces, stronger coding, or adaptive gating.

6.10. Rank Noise

We inject rank disagreement through the offline decoder proxy with noise rate ϵ, which lowers the per-bit match probability while keeping decoded length fixed. This is weaker than a live cross-model rank-reconstruction experiment but isolates the expected channel boundary. On ALFWorld, AsymAgentMark-TK maintains a bit-match rate of 0.900 at $\epsilon = 0.05$ and 0.858 at $\epsilon = 0.10$, degrading to 0.688 at $\epsilon = 0.30$. ToolBench starts near chance because of lower decoded length and capacity, but its degradation is smooth. The experiment confirms the expected boundary: rank-only decoding is insensitive to probability values but sensitive to rank disagreement.

6.11. Step Erasure

Step erasure models incomplete logs and early termination. Table 8 summarizes ALFWorld because it has enough trajectory length to expose the effect cleanly. Under exact probabilities, AsymAgentMark-TK degrades smoothly from 0.532 payload recovery with no erasure to 0.338 at 50% erasure. Under the harder top-6 rank view, bit match remains above chance and declines from 0.641 to 0.567, while strict payload recovery is low because the rank-only proxy produces too few equations. This separates the two bottlenecks: the rank channel still carries signal, but full payload recovery needs either longer traces, larger k, or stronger aggregation.

TABLE 7. RANK-NOISE SENSITIVITY FOR ASYMAGENTMARK-TK IN THE OFFLINE DECODER PROXY. CELLS SHOW BIT-MATCH / PAYLOAD RECOVERY.

Dataset	0.00	0.05	0.10	0.20	0.30
ALFWorld	.942/.532	.900/.368	.858/.239	.769/.108	.688/.042
ToolBench	.502/.033	.489/.025	.454/.017	.413/.003	.375/.003

TABLE 8. ALFWORLD ERASURE SENSITIVITY IN THE OFFLINE DECODER PROXY. CELLS SHOW BIT-MATCH / PAYLOAD RECOVERY.

Method	View	0.0	0.1	0.2	0.3	0.5
AgentMark-F	Exact	.985/.700	.976/.691	.969/.650	.944/.608	.915/.522
AgentMark-F	Top-6	.498/.004	.488/.003	.484/.004	.486/.001	.466/.002
AsymAgentMark-TK	Exact	.942/.532	.935/.506	.921/.482	.910/.431	.842/.338
AsymAgentMark-TK	Top-6	.641/.028	.631/.021	.614/.020	.602/.020	.567/.016

6.12. False Positives

False-positive control is analytic for strict payload claims. If a clean or unrelated trajectory yields coded bits that are independent of a pre-registered claimed payload, the chance that it recovers that exact L-bit payload is 2^{-L}. For payload lengths $L = 8, 16, 32, 64$, the corresponding single-claim thresholds are 3.91×10^{-3}, 1.53×10^{-5}, 2.33×10^{-10}, and 5.42×10^{-20}. Operational deployments should use at least 32-bit claims for audit-grade attribution, with longer payloads when enough trajectory length is available.

The threshold must also account for multiple testing. If an operator tests N independent key/payload claims against the same corpus, a conservative union bound gives a family-wise false-positive rate at most $N2^{-L}$. Thus an 8-bit payload is appropriate for controlled capacity diagnostics but not for open-ended attribution. A 64-bit payload keeps the bound below 10^{-12} even for one million tested claims, assuming the verifier also enforces the rank-quality gate described in Section IV-D.

The postprocessed false-positive artifact instantiates this calculation across dataset, model, split, and top-k configurations using 1000 random payload trials per clean trajectory. Because the strict event is exact payload equality, the per-claim rate equals the theoretical 2^{-L} in every cell; the expected count changes only with the number of tested trajectories and claims. This is why the paper reports false positives as an analytic security threshold rather than as a learned empirical classifier.

Bit-level scores require a different interpretation. When a verifier reports a match rate over n decoded coded bits rather than a recovered payload, the appropriate null is a binomial tail $\Pr[X \geq s; X \sim \mathrm{Binom}(n, 1/2)]$ for the observed score s. This statistic is useful for channel diagnostics and triage before enough independent RLNC equations are available, but it is not a positive attribution decision unless the pre-registered payload itself is recovered.

7. Discussion

Exact probability is a luxury. The evaluation supports a pragmatic conclusion: if exact P_t is available, symmetric

AgentMark-F can be more capacity-efficient. If the verifier has only top-k ranks, symmetric decoding becomes unreliable and weak asymmetry becomes necessary.

ToolBench exposes a low-entropy regime. Tool-use traces are shorter and often contain fewer semantically viable actions than embodied planning. AsymAgentMark-TK preserves utility in this setting, but the offline payload recovery remains low. This suggests combining rank encoding with adaptive step selection, longer audit windows, or higher-level tool-plan aggregation.

Rank consistency is observable. Unlike exact probability equality, rank consistency can be measured directly during verification. A verifier can report top-k agreement, Kendall tau distance, or adjacent-swap rates to calibrate confidence.

Adaptive adversaries. An adversary aware of the watermark could try to perturb ranks while preserving task success. This attack is outside pure distribution perturbation and should be treated as an active removal attack. Defenses include randomized k, context-dependent rank partitions, and cross-step consistency checks.

Operational audit flow. A practical verifier should not treat every low-confidence trajectory as a negative attribution. Instead, it should first check whether the top-k rank reconstruction is itself reliable. If the reconstructed rank agreement is poor, the correct conclusion is an inconclusive audit rather than a clean verdict. This distinction matters in legal or compliance settings, where absence of recoverable watermark evidence is not the same as evidence of absence.

From feasibility to deployment. The evaluation establishes the channel-knowledge tradeoff: rank-only verification can preserve useful signal when exact-probability verification is unavailable. A production deployment should add three engineering checks before relying on decoded payloads for attribution: live end-to-end RLNC recovery, calibrated rank reconstruction for the deployed verifier model or API, and pre-registered decision thresholds for positive, inconclusive, and negative outcomes. These checks are operational rather than conceptual, but they are important for turning weak-asymmetric provenance into an audit procedure.

8. Related Work

LLM and agent watermarking. Token-level LLM watermarking modifies or tests generated text distributions [4], [5]. Follow-up work studies reliability under paraphrasing, mixed documents, and edit-style perturbations [15], [16]. These methods are complementary to behavioral watermarking because agents often compile natural-language plans into structured actions, where the final text may no longer carry the decision signal. AgentMark [6] is the closest prior work and establishes distribution-preserving behavioral watermarking. AsymAgentMark-TK keeps its utility-preservation objective but weakens the verifier's channel requirement.

Steganography and distribution-preserving sampling. Provably secure steganography formalizes indistinguishability between cover and stego content [11]. Meteor provides a practical cryptographic construction for realistic distributions [12]. Weakly asymmetric steganography studies receivers with partial channel knowledge [7]. AsymAgentMark-TK adapts this idea from token/text channels to planning-behavior channels.

Agent benchmarks. ALFWorld aligns text-based embodied environments with interactive learning [8]. ToolBench evaluates LLM tool-use over real-world APIs [9]. OASIS demonstrates large-scale social-agent simulation [10]. These benchmarks highlight why behavior-level provenance matters: the action trajectory is often more important than the final message.

9. Threats to Validity

Offline postprocessing. Capacity and robustness experiments are computed from logged trajectories and offline decoder proxies. This is appropriate for isolating the channel-knowledge question, but it is weaker than a complete live deployment study with end-to-end RLNC recovery under production logging. The paper therefore labels proxy capacity claims explicitly and avoids treating them as full-system recovery guarantees.

Benchmark coverage. ALFWorld and ToolBench cover embodied planning and tool-use, but they do not exhaust the space of agentic workflows. Social simulation, GUI control, multi-agent collaboration, and long-running enterprise agents may have different rank stability and trajectory length. The construction is domain-agnostic, but the best k and payload length should be calibrated per environment.

Candidate canonicalization. Rank decoding assumes that the verifier can map the logged behavior and reconstructed candidates into the same canonical behavior set. This is straightforward for enumerated actions and structured tool calls, but less automatic for free-form plans, GUI operations, or APIs whose schemas evolve over time. A deployment should canonicalize tool names, arguments, and environment actions before ranking; otherwise a semantically correct but differently serialized behavior may appear outside the verifier's top-k list and become an erasure.

Distribution elicitation. The method assumes that the embedder can elicit or estimate a behavior distribution. If the candidate generator is poor, distribution preservation protects the wrong channel. This is a limitation inherited from behavioral watermarking generally, not a weakness specific to rank-only decoding.

Ethics considerations

This work is intended for provenance, audit, and authorized ownership verification. It should not be used to conceal malicious coordination or bypass platform policy. The construction assumes access to a behavior distribution at embedding time and top-k rank information at verification time. It does not protect against arbitrary rank manipulation, model-level watermark removal, or compromised keys. The

evaluation uses offline decoder artifacts; deployment use requires end-to-end RLNC recovery under live logging and cross-model rank reconstruction.

Watermarking creates governance risks as well as benefits. A false or overconfident attribution can harm users, developers, or organizations. Deployments should therefore publish the verification threshold, report inconclusive outcomes, retain audit logs, and separate watermark evidence from broader incident investigation. The method should not be used for covert monitoring of users without notice; its appropriate role is provenance for agents or services controlled by the embedding party.

10. Conclusion

Behavioral provenance for LLM agents should not depend on a verifier reproducing exact per-step probabilities. AsymAgentMark-TK shows that top-k rank order is enough to recover useful watermark signal while preserving the agent's marginal behavior policy. By replacing probability-dependent bins with a rank-decodable binary partition tree, the scheme trades some exact-channel capacity for robustness under realistic partial-channel verification. This weakly asymmetric view makes behavioral watermarking better aligned with how agents are actually audited.

These results should be read as evidence for partial-channel feasibility rather than as a final production certification. Exact-channel AgentMark-F remains attractive when the original behavior probabilities are available; AsymAgentMark-TK targets the common audit setting where only ranks can be reconstructed reliably. Closing the remaining deployment gap requires longer live trajectories, end-to-end coded-payload recovery, and cross-model rank calibration.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[3] Gemini Team, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[4] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, "A watermark for large language models," in *Proceedings of the International Conference on Machine Learning*, 2023.

[5] S. Dathathri, A. See, S. Ghaisas, P.-S. Huang, R. McAdam, J. Welbl, V. Bachani, A. Kaskasoli, R. Stanforth, T. Matejovicova, J. Hayes, N. Vyas, T. Zacchary, M. Zilka, I. Gabriel, W. Isaac, A. Guez, L. Weidinger, J. Mellor, D. Hassabis, K. Kavukcuoglu, G. Irving, and P. Kohli, "Scalable watermarking for identifying large language model outputs," *Nature*, 2024.

[6] K. Huang, J. Tan, Y. Wei, W. Li, Z. Zhang, H. Tian, Z. Yang, and L. Zhou, "AgentMark: Utility-preserving behavioral watermarking for agents," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2026.

[7] Anonymous, "From symmetry toward weak asymmetry: Secure steganography under the receiver's partial channel knowledge," *Manuscript*, 2026.

[8] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, "ALFWorld: Aligning text and embodied environments for interactive learning," in *Proceedings of the International Conference on Learning Representations*, 2021.

[9] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world apis," *arXiv preprint arXiv:2307.16789*, 2023.

[10] C. Gao, X. Lan, Z. Lu, J. Mao, J. Piao, H. Wang, D. Jin, and Y. Li, "OASIS: Open agents social interaction simulations with one million agents," *arXiv preprint arXiv:2411.11581*, 2024.

[11] N. J. Hopper, J. Langford, and L. von Ahn, "Provably secure steganography," in *Proceedings of the Annual International Cryptology Conference*, 2002.

[12] G. Kaptchuk, T. M. Jois, M. Green, and A. D. Rubin, "Meteor: Cryptographically secure steganography for realistic distributions," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2021.

[13] T. Ho, M. Médard, R. Koetter, D. R. Karger, M. Effros, J. Shi, and B. Leong, "A random linear network coding approach to multicast," *IEEE Transactions on Information Theory*, vol. 52, no. 10, pp. 4413–4430, 2006.

[14] DeepSeek-AI, "DeepSeek llm: Scaling open-source language models with longtermism," *arXiv preprint arXiv:2401.02954*, 2024.

[15] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, "On the reliability of watermarks for large language models," in *Proceedings of the International Conference on Learning Representations*, 2024.

[16] R. Kuditipudi, J. Thickstun, T. Hashimoto, and P. Liang, "Robust distortion-free watermarks for language models," *Transactions on Machine Learning Research*, 2024.